

Overflow: Algorithmic Tower Defense

Informe de análisis asintótico

Ashley Solano Alejandro Cubero
Kevin Velásquez

CI-0116 Análisis de Algoritmos y Estructuras de Datos
Escuela de Ciencias de la Computación e Informática, Universidad de Costa Rica

26 de septiembre de 2026

1. Metodología

1.1. Qué se mide

Cada torre guarda a los enemigos que tiene dentro de su radio en una de ocho estructuras de datos. Toda operación sobre la estructura (inserción, borrado o consulta) cuenta sus *pasos*: cada comparación, salto de puntero, corrimiento dentro de un arreglo y rotación de árbol vale un paso. Cada estructura anota sus pasos por categoría y devuelve exactamente la suma de lo que anotó, de modo que el costo que bloquea a la torre, el total de la bitácora y la suma de las cuatro categorías son siempre el mismo número.

Cada torre dispone de `STEPS_PER_TICK` pasos por tick, es decir, 2400 pasos por segundo de juego. En cada tick los gasta primero en las operaciones de mantenimiento pendientes (una cola FIFO) y, si la cola queda vacía, en a lo sumo un disparo. Una operación que cueste más que el presupuesto restante deja a la torre en deuda durante los ticks necesarios para pagarla.

1.2. Cómo es la carga

Dos propiedades del juego determinan en qué caso (mejor, promedio o peor) cae cada estructura, y se usan a lo largo del informe:

1. Los identificadores crecen con el orden de aparición, y los enemigos siguen todos la misma ruta a la misma velocidad. Por lo tanto entran al radio de una torre en orden creciente de identificador y **los más viejos, de identificador más chico, son los primeros en salir**.
2. El camino serpentea, de modo que cada ranura ve dos tramos de la ruta: un mismo enemigo puede salir del radio y volver a entrar más tarde, con un identificador menor que los que llegaron después.

1.3. Experimentos

Todos los datos provienen de partidas en modo sin ventana, cada experimento repetido con 30 semillas distintas (la semilla decide el orden en que aparecen los enemigos de cada oleada). En cada partida las ocho ranuras usan la misma estructura. La bitácora registra una fila por torre y por oleada.

Archivo	Configuración	Preguntas
normal_game.csv	8 estructuras, reglas normales	contexto
load_curve.csv	8 estructuras, -ignore-defeat	1 a 4
hive_hash.csv, hive_buckets.csv	tabla hash, solo Hive, función original y corregida	5
swarm_trees.csv	ABB y AVL, solo Swarm	6

Cuadro 1: Experimentos. **-ignore-defeat** sigue jugando aunque se acaben las vidas, para medir cada estructura bajo toda la curva de carga de las 20 oleadas en lugar de cortar la medición en la primera derrota.

Como contexto, la tabla 2 muestra hasta qué oleada llega cada estructura con las reglas normales y cuánto dura la partida. La tabla hash es la única que llega a la oleada 19; la lista ordenada es la primera en perder.

Estructura	Oleada alcanzada		Duración (s)		
	Media	Desv.	Media	Desv.	Rango
Lista ordenada	16,00	0,00	249	2,2	244–253
Arreglo ordenado	17,00	0,00	269	1,8	267–273
ABB	17,00	0,00	270	3,1	264–278
Lista enlazada	17,03	0,18	277	5,2	267–292
Montículo mínimo	17,40	0,50	284	8,4	271–297
AVL	18,00	0,00	294	2,3	290–300
Arreglo dinámico	18,27	0,45	302	7,8	293–318
Tabla hash	19,00	0,00	315	2,3	311–321

Cuadro 2: Partidas normales (30 semillas): oleada alcanzada y duración en segundos de juego.

La oleada alcanzada casi no varía entre semillas (desviación 0 en cinco de las ocho estructuras), pero eso no significa que las partidas sean iguales: la duración sí varía en todas. El ABB, por ejemplo, pierde siempre en la oleada 17, pero esa última oleada dura entre 10,0 y 20,0 segundos según la semilla. La oleada es una medida demasiado gruesa por dos razones. Cada oleada tiene un 30 % más de enemigos que la anterior, así que cuando una estructura empieza a saturarse la siguiente la desborda con mucho margen: la semilla cambia *cuánto* resiste la defensa en la última oleada, no *en cuál* cae. Además, la semilla solo decide el orden de aparición dentro de cada oleada; la cantidad de enemigos, sus categorías, su velocidad y su ruta son siempre las mismas. Las estructuras con desviación distinta de 0 (montículo, arreglo dinámico y lista enlazada) son las que quedan justo en el límite entre dos oleadas. Las preguntas siguientes usan medidas continuas (pasos, tamaños, tiempos), que sí reflejan la variación entre semillas.

2. Costo por operación contra tamaño del registro

Para cada fila de la bitácora se calcula el costo medio de una operación de mantenimiento (pasos de inserciones y borrados entre su número) y se lo compara con el tamaño medio del registro en esa oleada. Luego se ajusta por mínimos cuadrados ponderados (el peso es el número de operaciones de la fila) el modelo

$$c(n) = a + b f(n), \quad f(n) \in \{1, \log_2(n+1), n\},$$

usando para cada estructura la f que predice la teoría *para esta carga* (figura 1, tabla 3).

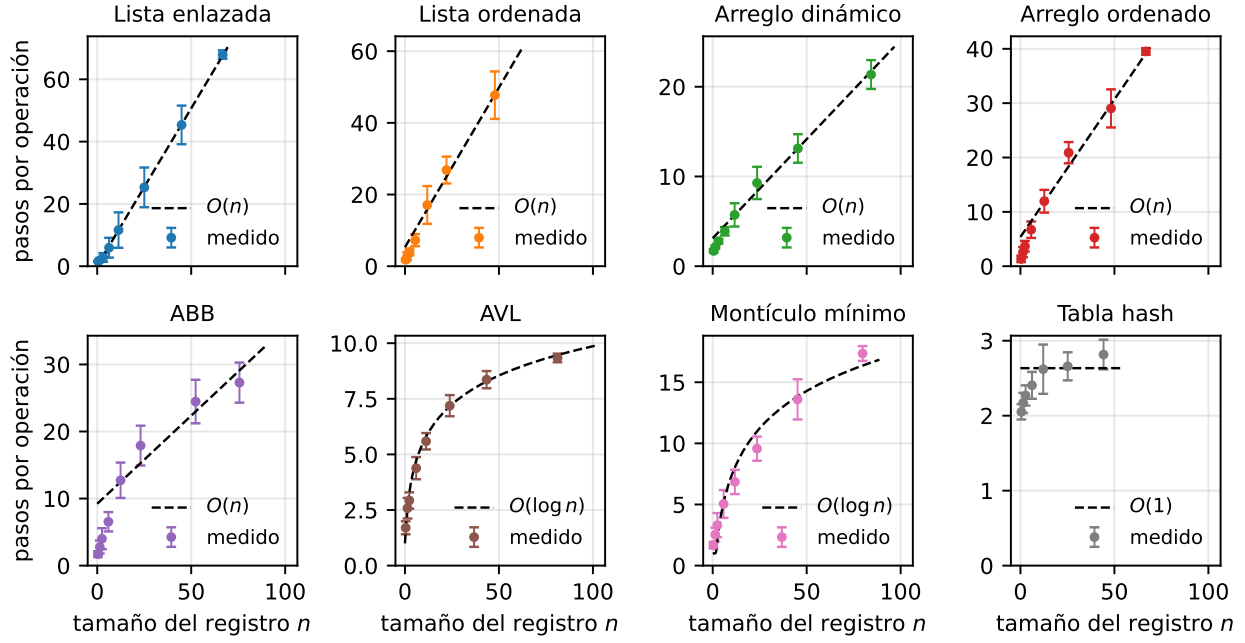


Figura 1: Costo medio por operación de mantenimiento según el tamaño del registro (puntos: media ponderada por rango de tamaños, barras: desviación estándar) y curva teórica ajustada.

Estructura	Modelo teórico	a	b	R^2 con $\log n$	R^2 con n
Lista enlazada	$O(n)$	0,12	1,011	0,810	0,983
Lista ordenada	$O(n)$	5,30	0,891	0,863	0,969
Arreglo dinámico	$O(n)$	3,11	0,221	0,830	0,964
Arreglo ordenado	$O(n)$	5,38	0,506	0,872	0,969
ABB	$O(n)$	9,22	0,263	0,862	0,755
AVL	$O(\log n)$	0,97	1,335	0,982	0,697
Montículo mínimo	$O(\log n)$	-3,50	3,135	0,908	0,903
Tabla hash	$O(1)$	2,63	—	0,413	0,298

Cuadro 3: Ajuste de $c(n) = a + b f(n)$ sobre 19 124 filas. Se muestra el R^2 de los dos modelos crecientes para comparar.

Estructuras lineales. La lista enlazada, la lista ordenada y los dos arreglos crecen de forma lineal, con R^2 entre 0,964 y 0,983. La pendiente refleja en qué caso cae cada una. La lista enlazada inserta al frente en 1,0 paso, pero el enemigo que sale es el más viejo, que está al final: cada borrado recorre la lista entera (61,1 pasos en promedio). Por eso su pendiente es $b \approx 1,011$, un paso por elemento. En la lista ordenada ocurre lo contrario: el identificador nuevo es el mayor, así que la inserción recorre hasta el final (47,0 pasos) y el borrado es más barato (22,8).

AVL y montículo. El AVL es logarítmico, como se espera ($R^2 = 0,982$). El montículo mínimo es el caso interesante: su borrado es una búsqueda lineal, y en teoría debería ser $O(n)$. Pero el enemigo que sale es el de identificador más chico, que es justamente la raíz: la búsqueda lo encuentra en el primer elemento y solo se paga el reacomodo, que es $O(\log n)$. La inserción de un identificador que siempre es el mayor tampoco sube nada (2,4 pasos). En esta carga el montículo es logarítmico ($R^2 = 0,908$). La ordenada negativa del ajuste ($a = -3,50$) es un artefacto de ajustar $\log_2(n + 1)$ cerca de $n = 0$.

Tabla hash. Es prácticamente constante: pasa de unos 2 pasos con registros vacíos a 2,8 con $n \approx 44$. El leve crecimiento no es ruido: una quinta parte de los enemigos son Hive, cuyos identificadores caen todos en la misma cubeta, y esa cubeta sí crece con n (sección 6).

Discrepancia: el ABB. Con identificadores crecientes el ABB degenera en una cadena, así que la teoría predice un costo lineal: insertar recorre la cadena entera (36,7 pasos) y borrar al más viejo, que es la raíz, casi no cuesta (4,3). Sin embargo, en la tabla 3 el modelo logarítmico ajusta mejor. Al separar las oleadas, la explicación aparece. Mientras la torre da abasto (cola vacía al menos el 95 % de los ticks), el costo es lineal ($R^2 = 0,920$ contra 0,889 del logarítmico). La curva solo se dobla en las oleadas saturadas, donde ninguno de los dos modelos explica los datos ($R^2 \approx 0,430$). En esas oleadas la cola retrasa las operaciones, de modo que el tamaño medio de la oleada deja de representar el tamaño que realmente ve cada operación. A esto se suma la reentrada de enemigos al radio, que inserta identificadores viejos por la izquierda y rompe la cadena (sección 7).

3. Punto de saturación

Una torre se satura cuando la cola de pendientes deja de vaciarse. Se define la saturación observada como la primera oleada en que la cola estuvo ocupada más de la mitad de los ticks, y el punto de saturación como el tamaño medio del registro en esa oleada.

Predicción. La oferta de operaciones de la torre es

$$S(n) = \frac{\text{STEPS_PER_TICK} \times \text{TICKS_PER_SECOND}}{c(n)} = \frac{2\,400}{c(n)} \text{ operaciones por segundo,}$$

con $c(n)$ el modelo ajustado en la sección anterior. La demanda la impone la rotación de enemigos: cada uno genera una inserción al entrar al radio y un borrado al salir. Midiendo solo las oleadas en que la torre daba abasto (y por lo tanto realizó toda la demanda), las operaciones por segundo son proporcionales al tamaño del registro: $D(n) = kn$ con $k = 5,81$ operaciones por segundo por enemigo en rango. El punto de saturación predicho n^* es el que iguala $S(n^*) = D(n^*)$.

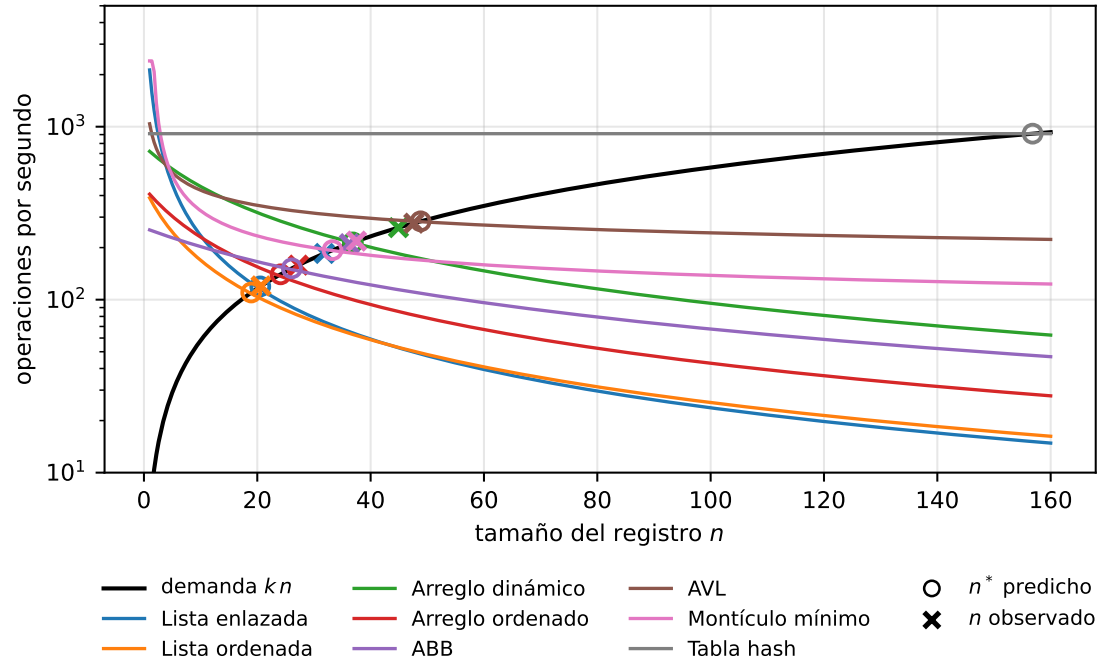


Figura 2: Oferta $S(n)$ de cada estructura contra la demanda $D(n)$. Los círculos marcan la saturación predicha y las cruces la observada (mediana).

Estructura	n^* predicho	n observado (mediana)	Rango intercuartil	Torres saturadas
Lista enlazada	21	32	26–45	240/240
Lista ordenada	19	21	19–25	240/240
Arreglo dinámico	37	45	32–48	240/240
Arreglo ordenado	24	27	26–31	240/240
ABB	26	36	28–49	240/240
AVL	49	48	46–50	180/240
Montículo mínimo	33	38	34–43	240/240
Tabla hash	157	—	—	0/240

Cuadro 4: Saturación predicha y observada. La última columna cuenta las torres (semilla y ranura) que llegaron a saturarse.

Resultados. La predicción ordena las estructuras igual que la observación, y en varias acierta casi exactamente: AVL (49 predicho, 48 observado), lista ordenada (19 y 21) y arreglo ordenado (24 y 27). La tabla hash tiene $n^* = 157$, por encima del registro más grande que se alcanzó en las partidas (101), y en efecto ninguna de sus torres se saturó.

Discrepancias. Donde la predicción falla, lo hace hacia el mismo lado: el tamaño observado es mayor que n^* (lista enlazada: 21 contra 32; ABB: 26 contra 36). Hay dos causas. La primera es la resolución: la saturación solo se observa por oleada, y como cada oleada es un 30 % más grande que la anterior, la oleada en que se detecta ya empezó por encima de n^* . La segunda es que $c(n)$ se ajustó con todas las oleadas, incluidas las saturadas, donde el costo real crece más despacio que el modelo (el caso del ABB en la sección anterior); eso subestima la oferta y adelanta la predicción. En sentido contrario actúa el presupuesto que la torre gasta en disparar, que la predicción ignora y que haría saturar antes.

4. Arreglo ordenado contra lista enlazada

El arreglo ordenado busca en $O(\log n)$ y la lista enlazada en $O(n)$. Aun así, el arreglo ordenado no supera a la lista: se satura antes (oleada 17,1 en promedio, contra 17,7) y en partidas normales llega a la misma oleada (17,00 contra 17,03).

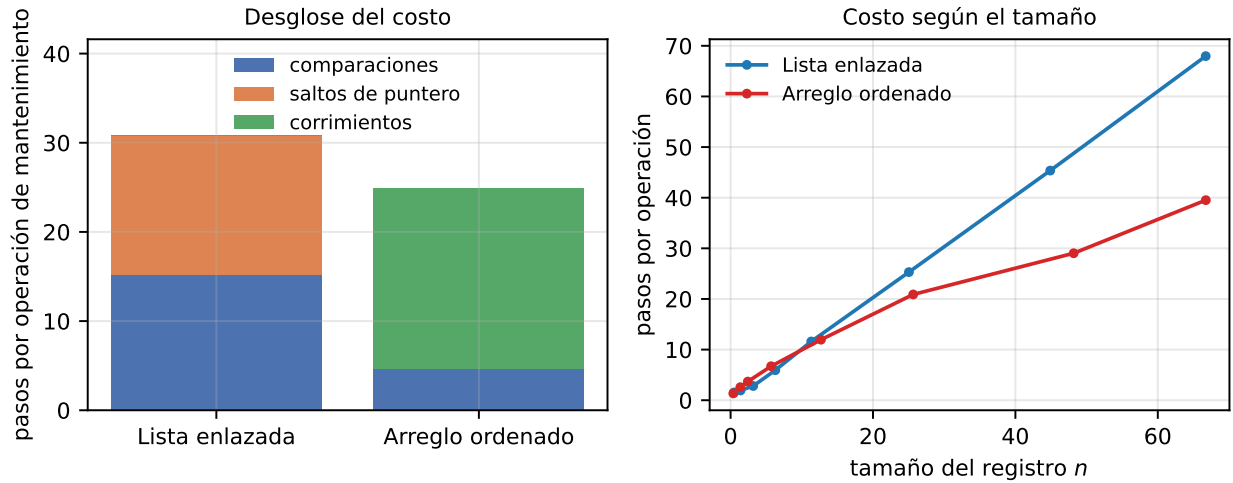


Figura 3: Izquierda: pasos por operación de mantenimiento según su categoría. Derecha: costo por operación según el tamaño del registro.

La búsqueda es lo de menos. La figura 3 muestra en qué gasta cada una. La búsqueda binaria cumple: el arreglo ordenado hace solo 4,70 comparaciones por operación, contra 15,16 de la lista. Pero para mantener el orden tiene que correr elementos en cada inserción y cada borrado, y esos corrimientos son $O(n)$: 20,21 por operación, el 81 % de todos sus pasos. Buscar en $O(\log n)$ no cambia que modificar cueste $O(n)$; las dos estructuras quedan en la misma clase asintótica. Solo difieren en la constante, y a favor del arreglo cuando n es grande (derecha de la figura).

Por qué igual pierde. La bitácora muestra la causa al comparar las dos estructuras ante la misma carga, es decir, la misma oleada (tabla 5). Las dos hacen exactamente los mismos disparos

efectivos hasta la oleada 16, pero el registro del arreglo ordenado es más del doble de grande (10,3 contra 3,9 en la oleada 16) y gasta casi cuatro veces más pasos (131 mil contra 35 mil). La diferencia está en *a quién* le dispara cada una. La consulta nativa de la lista devuelve la cabeza, que es el enemigo más nuevo: al matarlo, sale del registro un enemigo que se iba a quedar en el radio mucho tiempo. La del arreglo ordenado devuelve la posición 0, el identificador más chico, que es el enemigo más viejo, el que de todos modos estaba por salir del radio. Con los mismos disparos, el arreglo ordenado mantiene su registro más lleno, cada operación le cuesta más, y la cola se le llena antes.

Oleada	Lista enlazada				Arreglo ordenado			
	n	Cola vacía	Kpasos	Disparos	n	Cola vacía	Kpasos	Disparos
14	1,2	100 %	8,7	2 784	2,4	100 %	20,4	2 784
15	2,1	99 %	15,8	3 613	4,5	97 %	43,6	3 613
16	3,9	96 %	35,2	4 695	10,3	81 %	130,7	4 695
17	13,0	65 %	214,6	4 959	27,3	35 %	389,5	2 055

Cuadro 5: Misma oleada, lado a lado: tamaño medio del registro, fracción de ticks con la cola vacía, miles de pasos y disparos efectivos por partida (las 8 torres).

5. Pasos contados contra tiempo medido

Además de contar pasos, cada torre mide con el reloj del sistema el tiempo real que pasa dentro de cada operación. Si los pasos fueran una medida fiel del trabajo, el tiempo sería proporcional a ellos. No lo es, por dos razones que se separan con el modelo

$$t = \alpha m + \beta p,$$

donde m es el número de operaciones, p el de pasos, α un costo fijo por operación y β el costo de cada paso (tabla 6). El ajuste explica casi toda la variación ($R^2 \geq 0,95$ en las ocho estructuras).

Estructura	α (ns/op.)	β (ns/paso)	R^2	ns/paso promedio
Lista enlazada	74,5	3,39	0,972	6,8
Lista ordenada	73,8	3,74	0,950	6,4
Arreglo dinámico	72,8	2,87	0,983	12,0
Arreglo ordenado	86,5	4,85	0,975	9,1
ABB	70,6	4,37	0,989	8,6
AVL	48,7	24,56	0,994	31,5
Montículo mínimo	98,4	5,60	0,975	15,0
Tabla hash	100,3	24,68	0,995	39,4

Cuadro 6: Ajuste de $t = \alpha m + \beta p$ por estructura.

Con registros chicos domina el costo fijo. Cada operación cuesta entre 49 y 100 ns aunque no haga casi ningún paso. Es lo que no se cuenta como paso: la llamada virtual, las dos lecturas del reloj que miden la propia operación, y sobre todo la asignación dinámica, porque las listas y los árboles reservan un nodo con `new` en cada inserción y lo liberan con `delete` en cada borrado. Con pocos elementos las operaciones hacen uno o dos pasos, así que ese costo fijo, repartido entre ellos, da unos 50 ns por paso (izquierda de la figura 4). A medida que el registro crece, las operaciones

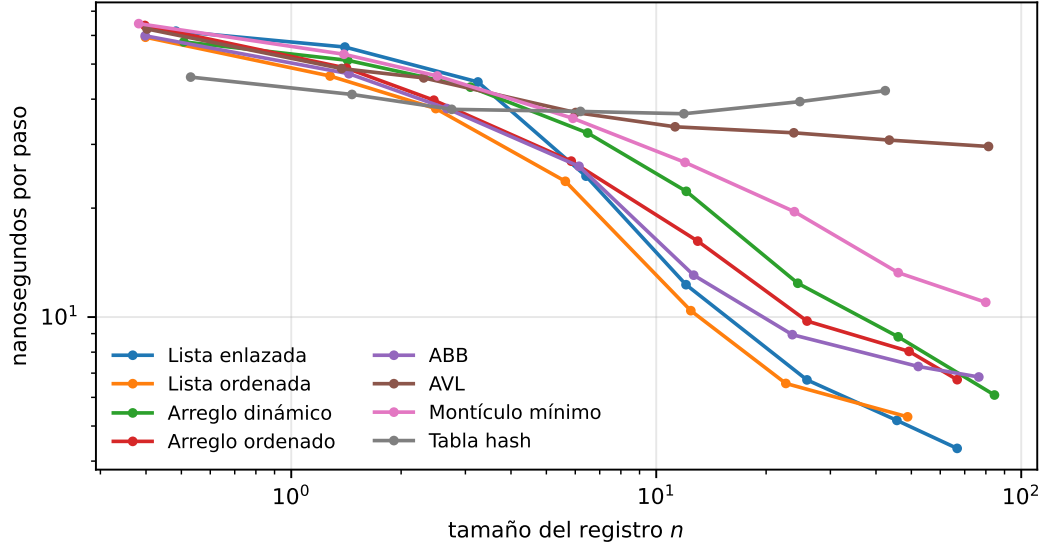


Figura 4: Nanosegundos por paso según el tamaño del registro (ejes logarítmicos).

hacen más pasos, el costo fijo se diluye y los nanosegundos por paso bajan: ahí es donde tiempo y pasos dejan de ser proporcionales.

No todos los pasos cuestan lo mismo. Con registros grandes, un paso de las listas y los arreglos cuesta entre 2,9 y 4,9 ns, pero uno del AVL o de la tabla hash cuesta unos 24,6 ns. El conteo de pasos trata igual una comparación de enteros que un paso del AVL, que en realidad incluye una llamada recursiva, el recálculo de alturas y la revisión del balance; o que un paso de la tabla hash, que incluye `std::hash`, una división entera para el módulo y el manejo de los `std::vector` de cada cubeta.

Jerarquía de memoria. Con los tamaños que alcanzó el juego (a lo sumo 101 elementos, unos pocos kilobytes) todo el registro cabe en la caché L1, de modo que en estos datos la jerarquía de memoria no explica las diferencias: recorrer una lista cuesta casi lo mismo por paso que recorrer un arreglo. Con registros de miles de elementos, que ya no caben en la caché, se esperaría lo contrario: cada salto de puntero de una lista o un árbol puede ser un fallo de caché, mientras que los corrimientos y las búsquedas en un arreglo contiguo aprovechan la precarga. Los arreglos ganarían ventaja por paso frente a las estructuras enlazadas.

Estos tiempos varían entre ejecuciones y dependen de la máquina; por eso el juego se rige por los pasos contados y no por el reloj.

6. Cubetas de la tabla hash bajo el Hive

Los identificadores de los Hive son múltiplos de 1024. La función de dispersión original es la identidad módulo el número de cubetas, que siempre es una potencia de dos no mayor que 1024, así que todos los Hive caen en la cubeta 0 sin importar cuántas veces la tabla crezca. La función corregida mezcla los bits del identificador (el finalizador de SplitMix64) antes de tomar el módulo. Se comparan las dos en partidas formadas solo por Hive (figura 5, tabla 7), tomando la tabla en su tamaño máximo de cada oleada y solo cuando tenía al menos 40 elementos.

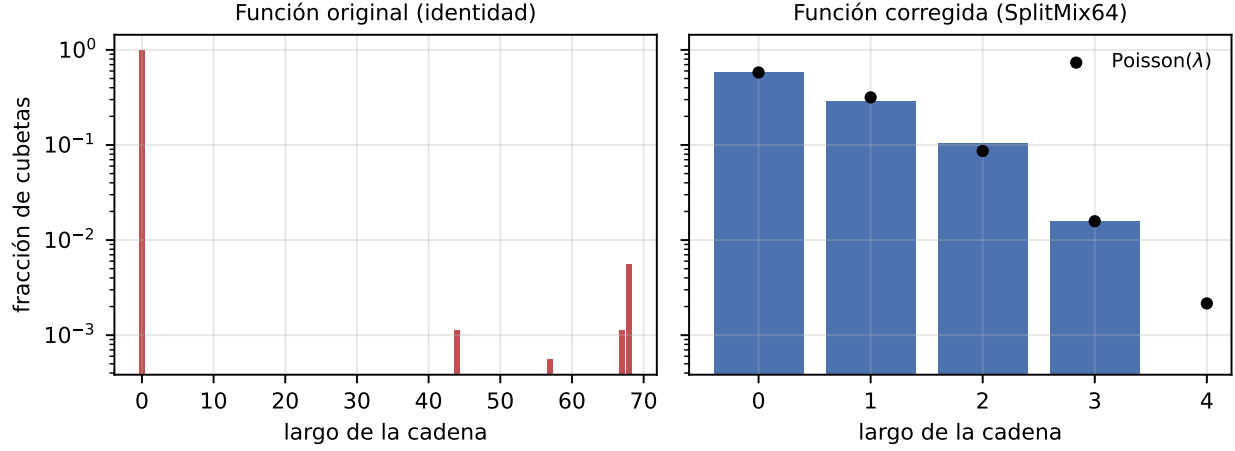


Figura 5: Distribución de largos de cadena (fracción de cubetas, escala logarítmica). A la derecha, los puntos son la distribución de Poisson que predice una dispersión uniforme.

Función	n máx.	Cubetas	Usadas	Cadena máx.	Pasos/op.	Cola vacía
Original (identidad)	68	128	1	68	37,3	52 %
Corregida (SplitMix64)	63	128	48	3	2,3	98 %

Cuadro 7: Tabla hash bajo el Hive: forma en el tamaño máximo alcanzado y costo por operación en todas las partidas.

Con la función original, el 99,2 % de las cubetas queda vacío y todos los elementos forman una sola cadena de hasta 68: la tabla degenera en una lista enlazada y cada operación cuesta $O(n)$, 37,3 pasos en promedio. Es un ataque de complejidad algorítmica: no hace falta ningún error en la estructura, basta con elegir claves que la función no sabe separar.

Con la función corregida, los mismos enemigos usan 48 de las 128 cubetas, ninguna cadena pasa de 3 y cada operación cuesta 2,3 pasos: 16,2 veces menos. La distribución coincide con la de Poisson de parámetro $\lambda = 0,55$ (elementos por cubeta) que predice una dispersión uniforme: 58,9 % de cubetas vacías contra 57,9 % esperadas, 29,2 % con un elemento contra 31,7 %, y 1,6 % con tres contra 1,6 %.

7. Altura del árbol bajo el Swarm

El Swarm llega en orden creciente de identificador. Se comparan el ABB sin balanceo y el AVL en partidas formadas solo por Swarm (figura 6, tabla 8), midiendo la altura cuando el árbol alcanza su tamaño máximo en cada oleada.

Árbol	n máx.	Altura máx.	Pasos/inserción	Pasos/borrado	Pasos/op.	Rotaciones/op.
ABB	88	57	52,6	2,8	27,87	0,00
AVL	142	8	7,1	6,4	6,73	0,59

Cuadro 8: ABB contra AVL bajo el Swarm.

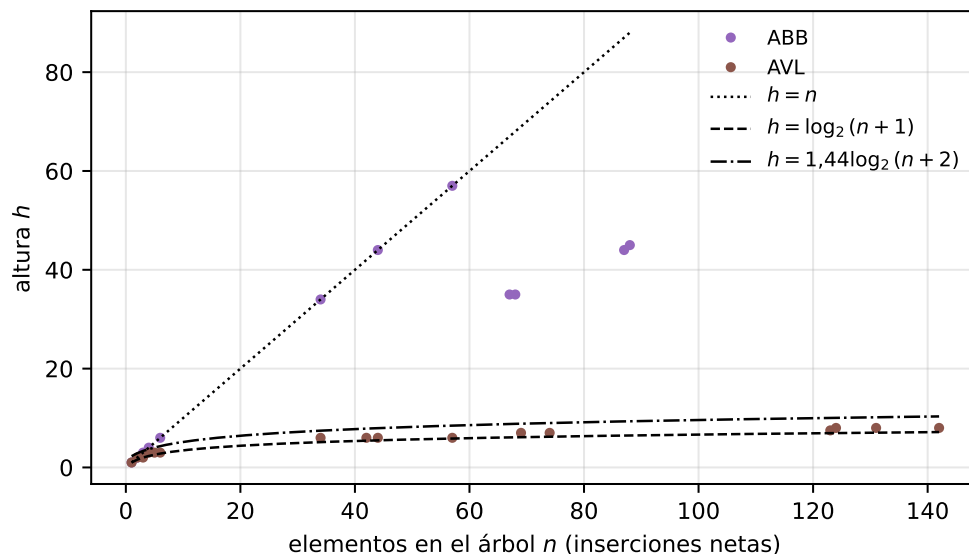


Figura 6: Altura del árbol según el número de elementos, con la altura de una cadena ($h = n$), la mínima posible ($\log_2(n + 1)$) y la cota superior del AVL ($1,44 \log_2(n + 2)$).

El ABB degenera. Cada identificador nuevo es mayor que todos los anteriores, así que se inserta como hijo derecho del último: el árbol es una cadena y su altura es exactamente n , como muestran los puntos sobre la recta $h = n$ hasta $n = 57$. Insertar recorre la cadena entera (52,6 pasos en promedio). Los puntos que caen por debajo de la recta con n grande se deben a la reentrada de enemigos al radio: un enemigo viejo que vuelve a entrar tiene un identificador menor que los recientes, se inserta por la izquierda y la cadena deja de ser pura.

El AVL se mantiene balanceado. Con hasta 142 elementos, su altura nunca pasa de 8, entre la altura mínima posible y la cota del AVL.

Costo de las rotaciones contra lo que ahorran. El AVL hace 0,59 rotaciones por operación, y cada una cuesta un paso. A cambio, una operación le cuesta 6,73 pasos contra 27,87 del ABB: ahorra 21,13 pasos por operación. Cada paso gastado en rotar ahorra unos 36 pasos de recorrido. El ABB solo es más barato al borrar (2,8 contra 6,4 pasos), porque el elemento más viejo es su raíz; el AVL, en cambio, paga al borrar el recorrido hasta el mínimo y el rebalanceo.

8. Conclusiones

El costo de cada estructura no depende solo de su complejidad en el libro, sino del caso en que la pone la carga. En este juego los enemigos entran en orden creciente de identificador y salen los más viejos primero: eso lleva a la lista enlazada, a la lista ordenada y al ABB a su peor caso, y al montículo a uno mejor que el general. El AVL y la tabla hash con una función de dispersión adecuada son las únicas estructuras cuyo costo casi no crece con n , y son las que llegan más lejos antes de saturarse. La fórmula de oferta y demanda predice bien el orden y, en varios casos, el tamaño exacto de la saturación. Por último, contar pasos es una medida reproducible y útil, pero no mide tiempo: el costo fijo de cada operación y el trabajo que esconde cada paso hacen que un paso de AVL o de tabla hash cueste cinco veces más que uno de una lista.

A. Reproducción

Desde la carpeta Proyecto1:

```
cd src && make && cd ..  
python3 tools/run_experiments.py      # 30 semillas -> report/data/  
python3 tools/make_figures.py         # figuras, tablas y números  
cd report && latexmk -pdf informe.tex
```

Las partidas son deterministas: la misma semilla produce la misma bitácora bit a bit, salvo la columna de tiempo real.